测试说明

> 对应赛项要求：「应提交可访问或可运行版本、系统说明、功能演示、界面截图、**技术架构和测试说明**」
>
> 全部结论来自**实际运行**，不是推测。测试环境见文末。

一、验证环境

项	值
运行时	Node.js v26.7.0
框架	Next.js 16.3.1 · React 19.2.8 · TypeScript 5.9.3 · Tailwind v4
模型	智谱 `glm-4-flash`（默认，服务端 Key）
服务	本地 `next dev -p 3100`，浏览器 `http://localhost:3100`
浏览器	ego lite（Chromium）
分支	`cmit-2026`（提交 `0bc0458` → `b2ffeb1`）

静态验证

- `tsc --noEmit` → 退出码 0
- `next build` → **Compiled successfully**，TypeScript 检查通过，8 个路由正常生成

二、v2 新增机制的验证

T1 · 悄悄话定向发言（`whisperOnly`）

项	内容
目的	确认悄悄话只唤起当事人，其余在座者不发言
输入	`characters: [luxun, steve-jobs-skill]`，`whisperOnly: "luxun"`，末条 user 消息带 `whisperTo: "luxun"`
预期	**只有 1 个 `who` 事件，且是 luxun**
实测	`who` 事件 1 个：`{"who":"luxun","name":"鲁迅"}` ✅

项	内容
结论	通过

T2 · 悄悄话内容不泄漏给非当事人

项	内容
目的	确认非当事人的 prompt 里不含悄悄话（服务端真过滤，不是前端隐藏）
方法	同 T1 的请求去掉 `whisperOnly`，两人都应发言
预期	两人都发言；只有收到悄悄话的人表现出"知道内情"
实测	`who` 事件 2 个（鲁迅 → 乔布斯）✅；鲁迅发言直接点名质问，乔布斯**主动辩解**而非附和
结论	通过。可见性由 `visibleHistory(speakerId)` 在**进入 prompt 之前**过滤

T3 · 摊牌机制真的触发（关键回归）

项	内容
背景	首版实现里，`mentionsSpeaker` 的右边界规则把「乔布斯**那套**」判成"没提到乔布斯"，导致摊牌指令**从未触发**——机制看起来装了，实际没动
修复	拆成两级匹配：严格（右边界非汉字，用于改发言顺序）+ 宽松（用于扫悄悄话在讲谁）
修复前实测	鲁迅回到「学外语的意义，绝不仅仅在于翻译与写作…」（泛泛而谈）❌
修复后实测	鲁迅：「**乔布斯曾言…那么我问他，改变世界后，你改变了多少人的思想？**」✅
结论	通过。这是本次开发中发现并修掉的最关键的一个 bug

T4 · 被点名者不再"服软"（抗伪共识）

项	内容
目的	被当众点名的人不许用「我同意」开场敷衍过去
修复前实测	乔布斯：「**我同意鲁迅的观点。**学外语，就像是拥有…」❌（典型的伪共识）
修复后实测	乔布斯：「**你说得对，我不是一个人做成的。但沃兹尼亚克也不是我雇来的**——他是因为看见我在做一件值得做的事才留下来的。」✅
结论	通过

T5 · 讨论停滞判据（信息新颖度）

项	内容
目的	用"信息新颖度"判定讨论是否在空转，并强制打破
输入	构造一段"上一轮原文重放"的历史（两条 assistant 消息与更早的两条完全一致）
预期	`novelty` 事件 `value` 接近 0、`stagnant: true`
实测	`{"type":"novelty","value":0,"stagnant":true}` ✅
连带效果	该轮发言由空话（「学外语仍然具有深远的意义」）转为**具体可操作的建议**
结论	通过

T6 · 悄悄话不污染查梗检索（泄漏修复）

项	内容
背景	v1 的查梗用「最后一条非 assistant 消息」触发；悄悄话若命中关键词，会被搜成「场外情报」并拼进**所有**发言者的 system——秘密直接公开
修复	触发源改为「最后一条**公开**的 user 消息」
实测	末条消息是悄悄话时，检索依据是**更早的那条公开消息**（返回的是 "Artificial intelligence (AI)" 而非悄悄话内容） ✅
结论	通过

T7 · 悄悄话不进日志

项	内容
方法	检查 `app/page.tsx` 提交给 `/api/log` 的 payload
实测	带 `whisperTo` 的消息在日志里被替换为 `content: "[whisper — content not logged]"`，只保留角色与目标 ✅
结论	通过

T8 · 主持人收尾（讨论趋同时拆穿伪共识）

项	内容
目的	讨论空转且强制反对无效时，由主持人出面指出问题
输入	一段"上一轮原文重放"的历史（新颖度 0%），两人圆桌
预期	发言顺序为「当事人甲 → 当事人乙 → 门外谈」，且主持人发言是**拆穿**而非总结
实测（第一版提示词）	❌ 主持人退化成复读机：「学外语，就是与另一个世界对话，就像鲁迅先生所言……」——把两人的观点又总结了一遍
实测（改进后）	✅ 「刚才这一轮等于什么都没谈成。你们只是重复了学习外语的好处，而没有指出真正的问题。目的不明确，方法不具体，真正的分歧绕开了，关键问题从第一句起就没人敢碰。」
结论	通过。**软提示无效，必须给新颖度数字 + 明确"不是总结是拆穿" + 一串禁令**

T9 · 生产环境部署验证（2026-10-17 19:03）

项	内容
目的	确认参赛版本真的在线上跑着，而不只是本地能跑
部署形态	阿里云香港 ECS · podman 容器 · systemd 托管（`Restart=always`）· nginx 反代 80/443
验证 1	`https://outgate.chat/` → **HTTP 200**；HTML 中出现 v2 独有标识（`rv-hero-ghost`、「私下说」按钮）
验证 2	容器内部 `127.0.0.1:3101` → **HTTP 200**；`systemctl is-active menwaitan` → **active**
验证 3（关键）	向生产接口 `POST https://outgate.chat/api/chat` 发送带 `whisperOnly: "luxun"` 的请求 → **只返回 1 个 `who` 事件**（鲁迅），其余在座者不发言 → 悄悄话定向发言在线上生效
验证 4	线上实跑一次完整会话：公开提问 → 两人公开发言 → 悄悄话 → 自动接公开轮 → 被点名者正面接火；英文模式聊天区汉字数 = 0
结论	通过。**线上即参赛版本，评审可直接访问验证**

三、v1 功能回归（确认 v2 没有弄坏既有能力）

#	用例	预期	实测	结果
R1	**单人对谈**（`characters` 只有 1 人，`mode: "chat"`）	走单人分支，不注入圆桌规则与信息层；不发 `novelty` 事件	`who` 事件 1 个、`novelty` 事件 0 个；回复「今天也辛苦了，休息一下，随便聊聊也行。」自然	✅

#	用例	预期	实测	结果
R2	**英文模式**（`lang: "en"`，两人圆桌）	两人都发言；回复不含汉字	`who` 事件 2 个（Lu Xun、Steve Jobs）；**汉字数量 0**	✅
R3	**中英双语界面**	悄悄话标记同步翻译	中文「▣ 只有 鲁迅 看得到」（纸白描边小方章，内写「密」）/ 英文「▣ Whispered — only Lu Xun can see this」	✅
R4	**12 人以上入席**	自动收缩布局，功能不受影响	`.rv-avatars mini` 生效，12 张卡片**每张仍带「私」按钮**（`document.querySelectorAll('.rv-avatar .wh').length === 12`）	✅
R5	**查梗**（公开消息命中热梗）	命中才搜，失败静默	`note` 事件正常返回	✅
R6	**SSE 流式**	逐字返回，`who` / `delta` / `done` / `end` 事件齐全	事件齐全，前端逐字渲染正常	✅
R7	**输出前缀清洗**	模型模仿历史格式产生的「某某说:」自我前缀不应出现在气泡里	修复前：`鲁迅说：乔布斯曾言…`；修复后：`乔布斯曾言…` ✅	✅

四、界面验证（截图见 `../screenshots/`）

在浏览器中实际操作确认的交互：

1. 点击输入框左侧「私下说」→ 选人 → 按钮变红显示「▣ 鲁迅」，输入框上方出现红色虚线状态条「▣ 正在私下对 鲁迅 说话 —— 桌上其他人看不到」，输入框提示文字变为「私下说点什么…」 ✅ （人物卡片上的「私」为快捷方式）
2. 发送后，该条消息在对话流中显示为深色气泡 + 红色虚线左边框 + 「▣ 只有 鲁迅 看得到」（纸白描边小方章，内写「密」） ✅
3. 悄悄话发送后**自动接一轮公开圆桌**，期间显示红条反白横幅「桌上 · 桌上正在反应 —— 你私下说的那件事，被人摆到台面上了」 ✅
4. 取消按钮可随时退出悄悄话状态 ✅
5. 入席人数降到 1 人、或悄悄话对象离席时，悄悄话状态自动清除 ✅

五、已知边界（如实说明，不回避）

项	说明
悄悄话一次只对一个目标	后端支持 `whisperTo` 传数组（多人可见），前端当前只做了"对一个人私下说"的入口。**这是有意收窄，不是能力缺失**
摊牌依赖模型配合	服务端给出的是**强指令 + 明确点名对象**，实测有效；但底层模型（glm-4-flash）偶有不完全执行的回合。**机制是真实的，效果有模型层面的波动**
新颖度阈值 0.34	基于实测调出的经验值。语料规模越大判定越稳；短对话（两三轮）样本少，判据仅供参考
讨论停滞后的动作	当前实现为「注入强制反对指令」。原设计里还有「换议题 / 拉新人 / supervisor 收尾总结」，**属后续迭代，未实装**
主持人收尾依赖模型	首版提示词下模型会退化成复读机；现版已实测有效，但强依赖提示词工程质量。这是**机制真实、效果有模型层面波动**的又一例
部署状态	**v2 已于 2026-10-17 19:03 部署到 https://outgate.chat**。本文的机制验证主要在本地 `next dev` 完成；部署后另做了一次生产验证（见 T9）

六、可复现步骤

```
# 1) 安装依赖并启动
pnpm install
pnpm dev -p 3100

# 2) 浏览器打开 —— 必须用 localhost
#    http://localhost:3100/
#    用 127.0.0.1 会被 Next.js 的跨域保护拦掉 JS chunk，页面能看但完全不可交互

# 3) 静态度量
npx tsc --noEmit          # 期望退出码 0
npx next build            # 期望 Compiled successfully

# 4) 接口级验证（悄悄话定向发言）
curl -X POST http://localhost:3100/api/chat \
  -H "Content-Type: application/json" \
  -d '{"characters":["luxun","steve-jobs-skill"],"mode":"debate","lang":"zh",
       "whisperOnly":"luxun",
       "messages":[{"role":"user","content":"你怎么看这件事？"},
                   {"role":"assistant","who":"乔布斯","content":"工具会变，品味不会。"},
                   {"role":"user","content":"乔布斯不赞同你的观点，你当面问
他。","whisperTo":"luxun"}]}'
# 期望：SSE 流中只出现 1 个 {"type":"who","who":"luxun"}
```